RJDAP – Reliable Job Dispatch & Aggregation Protocol

Phase 1 – Project Proposal

1. Assigned Scenario

RJDAP addresses a distributed job-dispatch scenario in which a central server divides or assigns computational jobs to worker clients. In the Phase 1 prototype, a worker receives a search task containing a start value, an end value, and a target value. The worker checks whether the target falls within the specified range and reports the outcome to the server.

The server is responsible for coordinating workers and dispatching tasks, while each worker is responsible for registering, receiving a task, performing the assigned computation, and returning a result. The design is intended to support multiple workers and provide a foundation for more reliable job handling in later phases.

2. Motivation

A distributed system can complete independent jobs by sharing work among several worker clients instead of relying on one machine to perform every computation. However, communication over UDP does not guarantee delivery, ordering, or duplicate protection. A task or result packet may be lost, and a sender may not know whether the receiver processed a message.

RJDAP therefore combines lightweight UDP communication with protocol-level mechanisms for identifying messages, detecting corrupted packets, acknowledging results, and handling retransmissions and duplicate results. The goal is to demonstrate a practical dispatch-and-result workflow while keeping the protocol small enough to implement and test locally.

3. Proposed Protocol Approach

3.1 Architecture and Communication

The system uses a client-server architecture. The server acts as the coordinator: it receives worker registration messages, assigns worker identifiers, and dispatches tasks. Worker clients initiate communication with the server using UDP, announce their availability, receive task assignments, execute the computation, and send results back.

The protocol distinguishes registration and control exchanges from task-data exchanges. A worker first sends a WORKER_JOIN message and receives a WORKER_ACCEPT response. The server then sends TASK_DISPATCH messages containing the task data. After completing the computation, the worker sends a TASK_RESULT message, and the server responds with ACK_RESULT. Workers also send HEARTBEAT messages to indicate that they remain active.

3.2 Message Format

RJDAP uses a fixed 16-byte message header followed by an optional payload. Multi-byte fields are encoded in network byte order (big-endian). The header identifies the protocol version and message type, associates the packet with a task and worker, records the attempt number and payload length, and carries a checksum.

Header Field	Size	Purpose
Version	1 byte	Identifies the protocol version
Message Type	1 byte	Identifies the message type
Task ID	4 bytes	Identifies the assigned task
Worker ID	2 bytes	Identifies the worker
Attempt Number	2 bytes	Distinguishes task attempts
Payload Length	2 bytes	Specifies the payload size
Checksum	4 bytes	CRC32 integrity check

The proposed maximum UDP application packet size is 512 bytes. The task payload contains three unsigned 32-bit integers: start, end, and target, making the payload 12 bytes in total. The worker checks whether the target is within the specified range, including both endpoints.

The result payload contains a one-byte status and a four-byte result value. Three status values are defined:

- SUCCESS (1): The target was found within the specified range.
- NOT_FOUND (2): The target was not found within the specified range.
- ERROR (3): An error occurred while processing the task.

3.3 Reliability and Worker Availability

Because UDP does not provide delivery acknowledgements, the protocol adds an application-level ACK_RESULT message. The worker matches an acknowledgement to the pending result using the Task ID, Worker ID, and Attempt Number. If an acknowledgement is not received, the worker can retransmit the same result message.

The server checks incoming results and prevents a repeated result for the same attempt from being stored more than once. If it receives a duplicate after the original result was accepted, it can send the acknowledgement again.

Workers periodically send HEARTBEAT messages. For the current design, the heartbeat interval is 2 seconds, and the server marks a worker inactive if no heartbeat is received for 6 seconds. If a worker repeats WORKER_JOIN from the same IP address and UDP port, the server returns its existing Worker ID rather than creating a second registration. This supports recovery when a registration response is lost.